ENG301

Timed Online Assignment – July Semester 2020

Microprocessor Programming

Tuesday, 10 November 2020 **4:00 pm – 6:30 pm**

Time allowed: 2.5 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **FOUR (4)** questions and comprises **TEN (10)** pages (including cover page).

2. You must answer **ALL** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. You are to include the following particulars in your submission: TOACourse Code, SUSS PI No., and Your Name. Name your submission file as TOACourseCode_FullName_StudentPI. Use underscore and NOT space.
   Example, TOAENG301_RaphaelLee_T1923161 (omit D/O, S/O).

6. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a Microsoft Word file saved in .doc or .docx format. All answers are to be typed in Times New Roman font, size 12 and single spacing. For answers which cannot be typed and require to be hand-written, ensure that question number is clearly stated on each page. Flowcharts and graphs may be scanned or photographed and embedded in the Word file in the right sequence provided it does not exceed the file size limit of 500 MB. All uploaded handwritten answers must be clear, readable and complete. Marks will not be awarded for un-readable or incomplete images.

7. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.

8. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

Answer all questions. (Total 100 marks)

Question 1

(a) Assembly language programming provides direct access to low-level processor functions and memory-sensitive data for precise control and efficient codes. Yet, high-level programming language, such as C/C++ remains popular and a common programming language for embedded system programing. Even so, graphical programming, such as LabView allows non-programmers to build embedded systems without the need to learn assembly language programming.

(i) Discuss **THREE (3)** advantageous features of high-level programing language, such as in C/C++ over assembly language programming.

(6 marks)

(ii) Examine the steps required to translate a graphical design in the form of a block diagram in LabView, and build it into machine codes to run on a target hardware.

(5 marks)

(b) ARM processor architecture supports both 32-bit ARM and 16-bit Thumb instruction sets.

(i) Appraise the motivation for mixing two instruction sets in a single ARM architecture, and the benefits derived from such combination.

(7 marks)

(ii) Discuss how ARM and Thumb instruction sets are intermixed and decoded to interwork freely on an ARM processor.

(7 marks)

Question 2

(a) The second operand in ARM data processing instruction has only a 12-bit field for specifying the value of an immediate constant.

(i) Analyse the encoding technique for representing a 12-bit number in a range of a 32-bit number. Criticise any limitation in such encoding technique.

(5 marks)

(ii) Demonstrate which of the following immediate constants can be encoded by the assembler. Provide reasons for those constants that cannot be successfully encoded.

- 0x101
- 0x102
- 0x103
- 0x104

(6 marks)

(b) Figure Q2(b) shows a basic ARM memory system with the relevant address lines and control signals. It is observed that the lower two address lines A[1:0] of the 32-bit address bus A[31:0] are not connected to the memory devices.

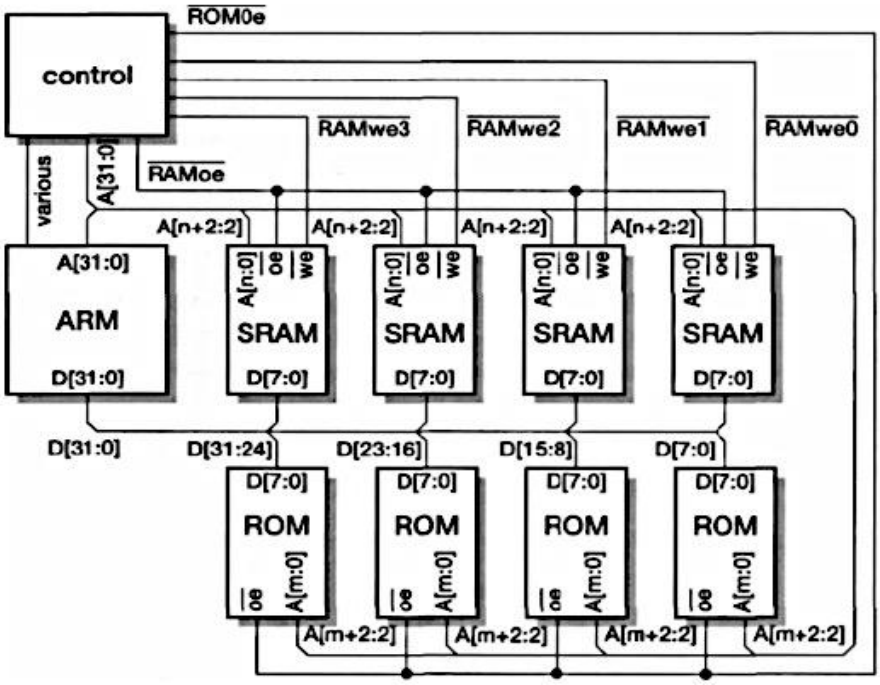

Figure Q2(b)

(i) Appraise the motivation for such memory interface design and judge any limitation it imposes on the addressability of a full 32-bit memory address space.

(4 marks)

(ii) The ROM and SRAM memory devices are connected to a common 32-bit data bus D[31:0] and share the same memory address space. Examine the control logic that allows the 32-bit memory address space to be shared between the ROM and SRAM devices.

(4 marks)

(iii) Suppose the ROM device has m = 30 address lines and that of the SRAM has n =16 address lines. Sketch the memory map and indicate the range of memory addresses for each of the devices.

(6 marks)

Question 3

(a) Figure Q3(a) shows a disassembled ARM program loaded in memory starting at location 0x280. Analyse the program and verify the contents in registers, r0 and r3 after the completion of program execution.

(9 marks)

Memory Location	Machine Code	Instruction/Data	
0x00000280	A005	ADR	r0,{pc}+0x18
0x00000282	F04F0103	MOV	r1,#0x03
0x00000286	F04F0200	MOV	r2,#0x00
0x0000028A	F8503B04	LDR	r3,[r0],#0x04
0x0000028E	1E49	SUBS	r1,r1,#1
0x00000290	441A	ADD	r2,r2,r3
0x00000292	DCFA	BGT	0x0000028A
0x00000294	E7FE	B	0x00000294
0x00000296	0000	DCW	0x0000
0x00000298	1234	DCW	0x1234
0x0000029A	0000	DCW	0x0000
0x0000029C	2345	DCW	0x2345
0x0000029E	0000	DCW	0x0000
0x000002A0	3456	DCW	0x3456
0x000002A2	0000	DCW	0x0000
0x000002A4	4567	DCW	0x4567
0x000002A6	0000	DCW	0x0000
0x000002A8	5678	DCW	0x5678
0x000002AA	0000	DCW	0x0000
0x000002AC	FFFFFFFF	DCD	0xFFFFFFFF

```
    AREA myCode, CODE, READONLY
    ENTRY
    EXPORT __main
__main
    ADR r0, Table
    MOV r1, #0x03        ; max loop count
    MOV r2, #0           ; accumulator
loop    LDR R3, [r0], #4
    SUBS r1, r1, #1
    ADD r2, r2, r3
    BGT loop
stop    B   stop
    ALIGN
Table
    DCD 0x1234, 0x2345, 0x3456, 0x4567, 0x5678
    END
```

Figure Q3(a)

(b) The following are individual arithmetic instructions, each operates on two register operands and stores its result in register r0:

```
ADDS  r0, r1, r1
ADDS  r0, r1, r2
SUBS  r0, r2, r1
SUBS  r0, r3, r2
```

Suppose that the values of the operands stored in registers r1, r2 and r3 are:

r1 = 0xFFFFFFFF
r2 = 0x80000000
r3 = 0x1

Verify which of the above instructions upon completion of execution will produce a result that matches one of the following conditions:

(i) A valid result with the carry bit set to 1
(ii) A valid result with the carry bit reset to 0
(iii) An invalid result

(Hints: Evaluate and examine the status of condition flags N, Z, C, and V).

(8 marks)

(c) The following 'while loop' written in C is an algorithm for computing the Greatest Common Divisor (GCD) between two integers, 'a' and 'b'.

```
while (a != b)
{
    if (a > b)
        a = a - b;
    else
        b = b - a;
}
```

Assume the values for 'a' and 'b' are stored in registers r0 and r1, respectively. The same loop in the GCD algorithm can be written in ARM assembly language as illustrated here:

```
loop
    CMP r0, r1
    BEQ end
    BLT less
    SUBS r0, r0, r1
    B loop      ; branch and loop
less
    SUBS r1, r1, r0
    B loop      ; branch and loop
end
```

However, the code is not efficient because of the number of branches. Re-write the GCD algorithm using conditional instructions to reduce the number of executions of loop and the code size.

(8 marks)

Question 4

(a) The assembly language program in Figure Q4(a) illustrates the use of a jump table to provide an efficient way to access and call one of a set of functions.

(i) Analyse the program and insert the missing codes to complete the program. You should also make the necessary correction to remove any erroneous codes. However, it should not be necessary to insert additional lines of codes to the existing program. (Hints: You should not rewrite the codes from Line 19 onwards and can be ignored in your answer.)

(12 marks)

```
1      AREA myData, DATA, READONLY
2  Num     EQU 0x1      ; pre-set number to select a function
3  OPD1    EQU 0x12     ; first operand
4  OPD2    EQU 0x24     ; second operand
5
6      AREA fnList, CODE, READONLY
7      ; a jump table for selection one of three functions
8      ENTRY
9      LDR r0           ; load the number to select a function
10     LDR r1           ; load the first operand
11     LDR r2           ; load the second operand
12     BL               ; ??
13 stop    B   stop
14
15 function
16     ADR r3           ; ??
17     LDR pc, [r3, r0, LSL #4]  ; ??
18
19 jumpTable
20     DCD Add, Subtract, Multiply
21
22 Add
23     ADD r4, r1, r2
24     BX lr
25
26 Subtract
27     SUBS r4, r1, r2
28     BX lr
29
30 Multiply
31     MUL r4, r1, r2
32     BX lr
33     END
34
```

Figure Q4(a)

(ii) Suppose that the program in Figure Q4(a) has been corrected of all code errors and successfully assembled. Execute the program and verify the value in the register r4 and the status of condition flags for N, Z, C, and V.

(5 marks)

(b) Design an ARM assembly language program to copy a block of data in multiple of **EIGHT (8)** words from one area of the memory to another. The data block consisting of 24 words and the address of the destination block are allocated memory areas with the following directives:

```
num  EQU    24    ; number of words to be copied
src  DCD    1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24
dst  EQU    0x20000000
```

To achieve code efficiency, you are required to use pre-indexed addressing with write-back in Increment After (IA) mode.

(8 marks)

APPENDIX: Reference Guide for ARM Instruction Set

Key to Tables	
{cond}	Refer to Table Condition Field {cond}
<Oprnd2>	Refer to Table Oprnd2
{field}	Refer to Table Field
S	Sets condition codes (optional)
B	Byte operation (optional)
H	Halfword operation (optional)
T	Forces address translation. Cannot be used with pre-indexed addresses
<a_mode1>	Refer to Table Addressing Mode 1
<a_mode2>	Refer to Table Addressing Mode 2
<a_mode3>	Refer to Table Addressing Mode 3
<a_mode4>	Refer to Table Addressing Mode 4
<a_mode5>	Refer to Table Addressing Mode 5
<a_mode6>	Refer to Table Addressing Mode 6
#32_Bit_Immed	A 32-bit constant, formed by right-rotating an 8-bit value by an even number of bits

Operation		Assembler	S updates	Action
Move	Move	MOV{cond}{S} Rd, <Oprnd2>	N Z C	Rd:= <Oprnd2>
	NOT	MVN{cond}{S} Rd, <Oprnd2>	N Z C	Rd:= 0xFFFFFFFF EOR <Oprnd2>
	SPSR to register	MRS{cond} Rd, SPSR		Rd:= SPSR
	CPSR to register	MRS{cond} Rd, CPSR		Rd:= CPSR
	register to SPSR	MSR{cond} SPSR{field}, Rm		SPSR:= Rm
	register to CPSR	MSR{cond} CPSR{field}, Rm		CPSR:= Rm
	Immediate to SPSR flags	MSR{cond} SPSR_f, 32_Bit_Immed		SPSR:= #32_Bit_Immed
	Immediate to CPSR flags	MSR{cond} CPSR_f, 32_Bit_Immed		CPSR:= #32_Bit_Immed
ALU	Arithmetic			
	Add	ADD{cond}{S} Rd, Rn, <Oprnd2>	N Z C V	Rd:= Rn + <Oprnd2>
	with carry	ADC{cond}{S} Rd, Rn, <Oprnd2>	N Z C V	Rd:= Rn + <Oprnd2> + Carry
	Subtract	SUB{cond}{S} Rd, Rn, <Oprnd2>	N Z C V	Rd:= Rn - <Oprnd2>
	with carry	SBC{cond}{S} Rd, Rn, <Oprnd2>	N Z C V	Rd:= Rn - <Oprnd2> - NOT(Carry)
	reverse subtract	RSB{cond}{S} Rd, Rn, <Oprnd2>	N Z C V	Rd:= <Oprnd2> - Rn
	reverse subtract with carry	RSC{cond}{S} Rd, Rn, <Oprnd2>	N Z C V	Rd:= <Oprnd2> - Rn - NOT(Carry)
	Multiply	MUL{cond}{S} Rd, Rm, Rs		Rd:= Rm * Rs
	accumulate	MLA{cond}{S} Rd, Rm, Rs, Rn,	N Z	Rd:= (Rm * Rs) + Rn
	unsigned long	UMULL{cond}{S} RdHi, RdLo, Rm, Rs	N Z	RdHi:= (Rm*Rs)[63:32] RdLo:= (Rm*Rs)[31:0]

Operation		Assembler	S updates	Action
	unsigned accumulate long	UMLAL{cond}{S} RdHi, RdLo, Rm, Rs	N Z	RdLo:=(Rm*Rs)+RdLo RdHi:=(Rm*Rs)+RdHi+ CarryFrom((Rm*Rs)[31:0]+RdLo))
	signed long	SMULL{cond}{S} RdHi, RdLo, Rm, Rs	N Z	RdHi:= signed(Rm*Rs)[63:32] RdLo:= signed(Rm*Rs)[31:0]
	signed accumulate long	SMLAL{cond}{S} RdHi, RdLo, Rm, Rs	N Z	RdHi:=signed(Rm*Rs)+RdHi+ CarryFrom((Rm*Rs)[31:0]+RdLo))
	Compare	CMP{cond} Rd, <Oprnd2>	N Z C V	CPSR flags:= Rn - <Oprnd2>
	negative	CMN{cond} Rd, <Oprnd2>	N Z C V	CPSR flags:= Rn + <Oprnd2>
	Logical			
	Test	TST{cond} Rn, <Oprnd2>	N Z C	CPSR flags:= Rn AND <Oprnd2>
	Test equivalence	TEQ{cond} Rn, <Oprnd2>	N Z C	CPSR flags:= Rn EOR <Oprnd2>
	AND	AND{cond}{S} Rd, Rn, <Oprnd2>	N Z C	Rd:= Rn AND <Oprnd2>
	EOR	EOR{cond}{S} Rd, Rn, <Oprnd2>	N Z C	Rd:= Rn EOR <Oprnd2>
	ORR	ORR{cond}{S} Rd, Rn, <Oprnd2>	N Z C	Rd:= Rn OR <Oprnd2>
	Bit Clear	BIC{cond}{S} Rd, Rn, <Oprnd2>	N Z C	Rd:= Rn AND NOT <Oprnd2>
	Shift/Rotate			See Table Oprnd2

Operation		Assembler	Action
Branch	Branch	B{cond} label	R15:= address
	with link	BL{cond} label	R14:=R15, R15:= address
	and exchange instruction set	BX{cond} Rn	R15:=Rn, T bit:= Rn[0]
Load	Word	LDR{cond} Rd, <a_mode1>	Rd:= [address]
	with user-mode privilege	LDR{cond}T Rd, <a_mode2>	
	Byte	LDR{cond}B Rd, <a_mode1>	Rd:= [byte value from address] Loads bits 0 to 7 and sets bits 8-31 to 0
	with user-mode privilege	LDR{cond}BT Rd, <a_mode2>	
	signed	LDR{cond}SB Rd, <a_mode3>	Rd:= [signed byte value from address] Loads bits 0 to 7 and sets bits 8-31 to bit 7
	Halfword	LDR{cond}H Rd, <a_mode3>	Rd:= [halfword value from address] Loads bits 0 to 15 and sets bits 16-31 to 0
	signed	LDR{cond}SH Rd, <a_mode3>	Rd:= [signed halfword value from address] Loads bits 0 to 15 and sets bits 16-31 to bit 15
	Multiple		
	Block data operations		
	Increment Before	LDM{cond}IB Rd{!}, <regs>{^}	Stack manipulation (pop)
	Increment After	LDM{cond}IA Rd{!}, <regs>{^}	
	Decrement Before	LDM{cond}DB Rd{!}, <regs>{^}	
	Decrement After	LDM{cond}DA Rd{!}, <regs>{^}	

Operation		Assembler	Action
	Stack operations	LDM{cond}<a_mode4>Rd{!}, <registers>	
	and restore CPSR	LDM{cond}<a_mode4> Rd{!}, <registers+pc>	
	User registers	LDM{cond}<a_mode4> Rd, <registers>^	
Store	Word	STR{cond} Rd, <a_mode1>	[address]:= Rd
	with user-mode privilege	STRT{cond} Rd, <a_mode2>	
	Byte	STRB{cond} Rd, <a_mode1>	[address]:= byte value from Rd
	with user-mode privilege	STRBT{cond} Rd, <a_mode2>	
	Halfword	STR{cond}H Rd, <a_mode3>	[address]:= halfword value from Rd
	Multiple		
	Block data operations		
	Increment Before	STM{cond}IB Rd{!}, <registers>{^}	Stack manipulation (push)
	Increment After	STM{cond}IA Rd{!}, <registers>{^}	
	Decrement Before	STM{cond}DB Rd{!}, <registers>{^}	
	Decrement After	STM{cond}DA Rd{!}, <registers>{^}	
	Stack operations	STM{cond}<a_mode5> Rd{!}, <regs>	
	User registers	STM{cond}<a_mode5> Rd{!}, <regs>^	
Swap	Word	SWP{cond} Rd, Rm, [Rn]	
	Byte	SWP{cond}B Rd, Rm, [Rn]	
Software Interrupt		SWI #24_Bit_Value	24-bit immediate value

Field	
Suffix	Sets
_c	Control field mask bit (bit 3)
_f	Flags field mask bit (bit 0)
_s	Status field mask bit (bit 1)
x	Extension field mask bit (bit 2)

Oprnd2	
Immediate value	#32_Bit_Immed
Logical shift left	Rm LSL #5_Bit _Immed
Logical shift right	Rm LSR #5_Bit _Immed
Arithmetic shift right	Rm ASR #5_Bit _Immed
Rotate right	Rm ROR #5_Bit _Immed
Register	Rm
Logical shift left	Rm LSL Rs
Logical shift right	Rm LSR Rs
Arithmetic shift right	Rm ASR Rs
Rotate right	Rm ROR Rs
Rotate right extended	Rm RRX

Addressing Mode 1	
Immediate offset	[Rn, #+/-12_Bit_Offset]
Register offset	[Rn, +/-Rm]
Scaled register offset	[Rn, +/-Rm, LSL #shift_imm]
	[Rn, +/-Rm, LSR #shift_imm]
	[Rn, +/-Rm, ASR #shift_imm]
	[Rn, +/-Rm, ROR #shift_imm]
	[Rn, +/-Rm, RRX]
Pre-indexed offset	
Immediate	[Rn, #+/-12_Bit_Offset]!
Register	[Rn, +/-Rm]!
Scaled register	[Rn, +/-Rm, LSL #shift_imm]!
	[Rn, +/-Rm, LSR #shift_imm]!
	[Rn, +/-Rm, ASR #shift_imm]!
	[Rn, +/-Rm, ROR #shift_imm]!
	[Rn, +/-Rm, RRX]!
Post-indexed offset	
Immediate	[Rn], #+/-12_Bit_Offset
Register	[Rn], +/-Rm
Scaled register	[Rn], +/-Rm, LSL #shift_imm
	[Rn], +/-Rm, LSR #shift_imm
	[Rn], +/-Rm, ASR #shift_imm
	[Rn], +/-Rm, ROR #shift_imm
	[Rn, +/-Rm, RRX]

Addressing Mode 2	
Immediate offset	[Rn, #+/-12_Bit_Offset]
Register offset	[Rn, +/-Rm]
Scaled register offset	[Rn, +/-Rm, LSL #shift_imm]
	[Rn, +/-Rm, LSR #shift_imm]
	[Rn, +/-Rm, ASR #shift_imm]
	[Rn, +/-Rm, ROR #shift_imm]
	[Rn, +/-Rm, RRX]
Post-indexed offset	
Immediate	[Rn], #+/-12_Bit_Offset
Register	[Rn], +/-Rm
Scaled register	[Rn], +/-Rm, LSL #shift_imm
	[Rn], +/-Rm, LSR #shift_imm
	[Rn], +/-Rm, ASR #shift_imm
	[Rn], +/-Rm, ROR #shift_imm
	[Rn, +/-Rm, RRX]

Condition Field {cond}	
Suffix	Description
EQ	Equal
NE	Not Equal
CS	Unsigned higher or same
CC	Unsigned lower
MI	Negative
PL	Positive or zero
VS	Overflow
VC	No overflow
HI	Unsigned higher
LS	Unsigned lower or same
GE	Greater or equal
LT	Less than
GT	Greater than
LE	Less than or equal
AL	Always

Addressing Mode 3 – Signed Byte and Halfword Data Transfer	
Immediate offset	[Rn, #+/-8_Bit_Offset]
Pre-indexed	[Rn, #+/-8_Bit_Offset]!
Post-indexed	[Rn], #+/-8_Bit_Offset
Register	[Rn, +/-Rm]
Pre-indexed	[Rn, +/-Rm]!
Post-indexed	[Rn], +/-Rm

Addressing Mode 4	
Addressing Mode	Stack Type
IA Increment After	FD Full Descending
IB Increment Before	ED Empty Descending
DA Decrement After	FA Full Ascending
DB Decrement Before	EA Empty Ascending

Addressing Mode 5	
Addressing Mode	Stack Type
IA Increment After	EA Empty Ascending
IB Increment Before	FA Full Ascending
DA Decrement After	ED Empty Descending
DB Decrement Before	FD Full Descending

----- END OF PAPER -----